# 30 分Consistency：强制内存顺序深度解析

作者：Ethan Zhang

Ethan Zhang
Software Engineer
Tencent Cloud



## 开篇思考

在多核处理器的世界里，内存顺序（Memory Ordering）决定了一个核心上的写入操作，其他核心能以什么顺序看到这些变化。默认情况下，CPU 会为了性能对指令进行重排，这可能会导致多线程程序出现意想不到的错误。

## 内存顺序的分类

C++ 提供了 6 种内存顺序，从最宽松到最严格分别是：

```
enum memory_order {
    memory_order_relaxed,
    memory_order_consume,
    memory_order_acquire,
    memory_order_release,
    memory_order_acq_rel,
    memory_order_seq_cst
};
```

本节课以 Consistency 为例，讲解 Release-Acquire 模型。Release 对应生产者，Acquire 对应消费者，两者配合可以保证生产者在 release 之前的所有写入，对消费者在 acquire 之后的操作可见。

## 生产者-消费者模型

```
int data = 0;
int ready = 0;

void producer() {
    data = 42;
    ready = 1;
}

void consumer() {
    while (ready == 0) {
        ;
    }
    int x = data;
    printf("%d", x);
}
```

如果没有内存顺序保证，编译器或 CPU 可以把 ready = 1 提前到 data = 42 之前，消费者就会读到 ready == 1 但 data == 0 的情况。

## 使用 Release-Acquire 修复

```
int data = 0;
int ready = 0;

void producer() {
    data = 42;
    ready.store(1, memory_order_release);
}

void consumer() {
    while (ready.load(memory_order_acquire) == 0) {
        ;
    }
    int x = data;
    printf("%d", x);
}
```

release 保证 data = 42 不会被重排到 ready.store 之后，acquire 保证 ready.load 之后的读取不会被重排到之前。

## 内存屏障

内存屏障（Memory Barrier）是实现内存顺序的底层机制。

```
void producer() {
    data = 42;
    memory_barrier();
    ready = 1;
}
```

## 编译器 vs 硬件

内存顺序需要同时约束编译器和硬件。

```
int data = 0;
int ready = 0;

void producer() {
    data = 42;
    ready.store(1, memory_order_release);
}

void consumer() {
    while (ready.load(memory_order_acquire) == 0) {
        ;
    }
    int x = data;
    printf("%d", x);
}
```

## 编译器重排

编译器会在不改变单线程语义的前提下，对指令进行重排以优化性能。

```
int a = 0;
int b = 0;

void foo() {
    a = 1;
    b = 2;
    int x = a + b;
    printf("%d", x);
}
```

## 硬件重排

即使编译器不重排，CPU 也可以在运行时乱序执行指令。

```
int data = 0;
int ready = 0;

void producer() {
    data = 42;
    ready = 1;
}

void consumer() {
    while (ready == 0) {
        ;
    }
    int x = data;
    printf("%d", x);
}
```

## 内存屏障的开销

内存屏障会限制编译器和 CPU 的优化空间，因此会带来性能开销。

```
void producer() {
    data = 42;
    memory_barrier();
    ready = 1;
}
```

## 内存顺序的性能影响

```
int data = 0;
int ready = 0;

void producer() {
    data = 42;
    ready.store(1, memory_order_release);
}

void consumer() {
    while (ready.load(memory_order_acquire) == 0) {
        ;
    }
    int x = data;
    printf("%d", x);
}
```

## 总结

- 默认情况下，编译器和 CPU 会为了性能对指令进行重排，这可能会导致多线程程序出现意想不到的错误。
- 内存顺序分为 6 种，从最宽松到最严格分别是：relaxed、consume、acquire、release、acq_rel、seq_cst。
- 本节课以 Consistency 为例，讲解 Release-Acquire 模型。Release 对应生产者，Acquire 对应消费者，两者配合可以保证生产者在 release 之前的所有写入，对消费者在 acquire 之后的操作可见。
- 内存屏障是实现内存顺序的底层机制，它会限制编译器和 CPU 的优化空间，因此会带来性能开销。
- 内存顺序需要同时约束编译器和硬件，编译器会在不改变单线程语义的前提下，对指令进行重排以优化性能；即使编译器不重排，CPU 也可以在运行时乱序执行指令。

关注我看时间服务号
每日学习签到
关注公众号，回复 Time，开启每日学习之旅

欢迎关注我的公众号：Ethan Zhang
公众号内容：分享技术、生活、思考
公众号名称：Ethan Zhang
公众号ID：ethan-zhang-1992

Copyright © 2025 Ethan Zhang. All rights reserved.

Powered by Gitalk

Powered by Giscus